计算机组织与结构
复习笔记

为北邮软工期末复习而作

Nam1dAme

2026 年 1 月 20 日

前言

本笔记的写作工作启动于 2025 年 11 月 22 日，正式版发布于 github 。本笔记适用于北京邮电大学软件工程专业大二课程《计算机组织与结构》。该课程使用教材为《Computer Organization and Architecture: Designing for Performance》, William Stallings 等著。考虑到该书内容过于庞杂，本笔记仅选取部分章节。需要注意的是，在本课程的实际教学中，教学内容和课件均基于第 8 版教材，而课后习题选自第 10 版，尽管第 10 版新增的章节不在考察范围内；因此，本笔记的章节号将以第 8 版教材以及课件为准，而课后习题的题号将以第 10 版为准。

本笔记的附录中包含了软工专业 24 级的期中和期末考题，均源自笔者考后回忆，不能保证与原文完全一致，但可以通过题意找到类似的习题。

笔者不会用 LaTeX 或 Typst，排版难免有拙劣之处，还望读者谅解。笔者水平有限，无法做到尽善尽美，若笔记内容与教材内容或课件有所出入，还请以老师以及教学实际为准，欢迎细心的读者通过 issue 或 邮件 予以斧正。有任何修改意见也欢迎联系笔者，尽管本笔记不太可能有大更新。将来的某一天也许会补上缺失的章节。

最后，感谢您的阅读。愿您在期末取得理想的成绩。

Nam1dAme

目录

前言 ..... **i**
目录 ..... **ii**
2. 性能问题 **1**
2.1 性能设计 1
2.2 Amdahl 定律 2
2.3 计算机性能基本测量 2
4. 高速缓存 **5**
4.1 计算机存储系统概述 5
4.2 Cache 原理 6
4.3 Cache 设计要素 6
7. 输入 / 输出 **10**
7.1 外部设备分类 10
7.2 I/O 模块 10
7.3 编程式 I/O 11
7.4 中断式 I/O 11
7.5 直接存储器访问 12
7.6 直接高速缓存访问（第 10 版新增） 13
7.7 I/O 通道和处理器 13
9. 计算机算术 **14**
9.1 算术逻辑单元 14
9.2 整数的表示 14
9.3 整数的运算 15
9.4 浮点数的表示 15
9.5 浮点数的运算 17
10.指令集：特性与功能 **18**
10.1 机器指令特征 18
10.2 操作数类型 18
10.3 操作类型 19
10.4 端序支持 20
11. 指令集：寻址模式与格式 **21**
11.1 寻址 21
11.2 指令格式 23
12. 处理器结构与功能 **24**
12.1 & 12.2 处理器与寄存器组成 24
12.3 指令周期 25

12.4 指令流水线 26
12.5 流水线冒险 28
13. 精简指令集计算机 30
13.1 寄存器和指令集 30
13.2 精简指令集架构 30
13.3 大寄存器组的使用 31
13.4 基于编译器的寄存器优化 32
13.5 RISC 流水线 33
13.6 RISC 和 CISC 的争议 34
14. 指令级并行与超标量处理器 36
14.1 超标量综述 36
14.2 超标量设计问题 38
附录 A: 24 级期中试题回忆 40
附录 B: 24 级期末试题回忆 41

2. 性能问题

2.1 性能设计

摩尔定律 Moore's theorem

通过在芯片上集成更多的晶体管和缩短电路之间的距离，微处理器的性能每三年可以提高 4-5 倍；动态随机访问存储器 (DRAM) 的容量每三年增加 4 倍。

提高微处理器的速度的常见技术包括：流水线、分支预测、数据流分析、推测执行、超标量执行。

存储器解决方案

CPU 的性能高速发展的同时，内存的性能也会影响 CPU 性能的发挥。 提高内存性能的方案：

- 增加一次访问的位数，使 DRAM 更宽而不是更深
- 在 DRAM 芯片中包含 Cache 来改变 DRAM 的接口
- 使用更复杂的 Cache 和片上 Cache ，减少存储器的访问频率
- 提高互联的带宽：高速总线和多级总线

I/O 设备解决方案

I/O 设备种类多，数据速率差异大。

解决方案：缓存 (Cache)、缓冲 (Buffer)、高速互联总线、更精巧的总线结构、I/O 通道和处理器

处理器速度的解决方案

1. 提高处理器硬件速度

   提高芯片集成度和时钟频率，以实现性能提升，但会带来**能耗**和**容阻迟滞**问题。

   - 逻辑密度和时钟速度的增加会导致功率密度的增加，散热成为设计问题。
   - 芯片上组件尺寸越小，导线越细，电阻越大；导线越近，电容越大。延迟也随之增加。

2. 提高 Cache 的大小和速度

   内存的速度滞后于处理器速度，因此处理器和内存间存在**缓存**。随着芯片密度增加，更多 Cache 被集成到芯片上，从而加快了访问速度。

3. 改变处理器的组织架构

   - **流水线**：让不同指令的不同阶段在流水线中同时发生。
   - **超标量**：允许单个处理器有多条流水线，相互独立的指令可以并行执行。

改进性能的新方法

然而，时钟频率越快，功耗越大，已经达到了一些基本的物理极限；通过 Cache 得到的收益也到了极限；增加组织架构的复杂度能获得大量的并行性，但要取得显著进步越来越难。

1. **多核处理器**

单个芯片上使用多个处理器，即多核。单个处理器中，性能的提高与复杂度增加的平方根成正比；但如果软件可以使用多个处理器，处理器数量翻倍能使性能翻倍。

2. **体系结构优化**

- **CISC** (Complex Instruction Set Computer) 即复杂指令集，典型代表为 Intel x86，用于个人计算机、服务器和超级计算机等。
- **RISC** (Reduced Instruction Set Computer) 即精简指令集，典型代表为 ARM，用于嵌入式系统、手机、MIPS 等。

3. **通用 GPU** (图形处理单元)

GPU 对图形数据进行**并行**处理，经常用作向量处理器，也用于需要重复计算的应用，比如模型训练。其数据处理性能远大于传统 CPU。

GPGPU (GPU 上的通用计算) 利用处理图形任务的图形处理器，计算原本由 CPU 处理的通用计算任务。

2.2 Amdahl 定律

符号定义：

f：代码中可无限**并行**执行的指令比例

$1-f$：需要**串行**执行的比例

T：在单个处理器上的执行时间

N：处理器数量

Amdahl 定律：

$$\begin{aligned}\text{加速比} &= \frac{\text{程序在单处理器上的执行时间}}{\text{程序在 } N \text{ 个并行处理器上的执行时间}} \\ &= \frac{T(1-f)+Tf}{T(1-f)+\dfrac{Tf}{N}} = \frac{1}{(1-f)+\dfrac{f}{N}}\end{aligned}$$

当 f 很小时，并行加速器的使用效果不大；当 N 趋于正无穷时，加速比的极值为 $\frac{1}{1-f}$，使用更多处理器的收益会递减。

2.3 计算机性能基本测量

时钟速度

处理器速度由时钟产生的脉冲频率决定，单位为赫兹 (Hz)。脉冲速率就是**时钟频率**或**时钟速度**，时钟的一个脉冲就是一个**时钟周期**。

指令的执行涉及多个离散的步骤，比如取指、解码、加载、存储、算术逻辑运算，因此指令通常需要多个时钟周期完成。流水线技术可以同时执行多个指令，所以不完全是串行过程，时钟速度也不能完全代表性能。

指令执行速度

符号定义：

f：时钟频率

τ：恒定时间，即 $\tau = \dfrac{1}{f}$

I_c：指令条数

每条指令平均周期数 CPI (Cycle Per Instruction) 计算方法：

$$\mathrm{CPI} = \frac{\sum_{i=1}^{n}(\mathrm{CPI}_i \times I_i)}{I_c}$$

执行给定程序所需的时间 T 表示为：

$$T = I_c \times \mathrm{CPI} \times \tau$$

每秒百万指令数 MIPS (Million Instruction Per Second) 计算方法：

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{\mathrm{CPI} \times 10^6}$$

每秒百万条浮点型指令数 MFLOPS：

$$\mathrm{MFLOPS} = \frac{\text{程序中浮点操作数量}}{\text{执行时间} \times 10^6}$$

基准测试程序

MIPS 仅反应机器执行的效率，而指令是并行执行的，不同机器对不同类型的指令序列进行了优化。由于指令集不同，指令执行速率不宜用于比较不同架构性能，不能仅用 MIPS 评价处理器性能。**基准测试包**是用高级语言编写的程序集，其中最著名的是由 SPEC (标准性能评估) 公司定义的 SPEC CPU2006，是适用于测试计算密集型应用的行业标准包。对于测试结果的算术处理，有三种平均方式：

1. 算术均值

$$\mathrm{AM} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

2. 调和均值

$$\mathrm{HM} = \frac{n}{\sum_{i=1}^{n} \times \dfrac{1}{x_i}}$$

3. 几何均值

$$\mathrm{GM} = \sqrt[n]{x_1 \times \cdots \times x_n} = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}} = \mathrm{e}^{\left(\frac{1}{n}\sum_{i=1}^{n} \ln(x_i)\right)}$$

对于几种均值，有以下不等式成立，且 $x_1 = x_2 = \cdots = x_n$ 时三者相等。

$$\mathrm{AM} \leq \mathrm{GM} \leq \mathrm{HM}$$

4. 高速缓存

4.1 计算机存储系统概述

存储系统的特性

- **位置**
  - 内部：寄存器、 Cache 、主存。
  - 外部：磁盘、磁带、 CD-ROM 。需要通过 I/O 控制器访问。
- **容量**
  - 单位：字节 (1 byte = 8 bit) 或字 (word) 。一个字可能是 16 位、32 位或 64 位。
  - 可寻址单元：内存中能唯一寻址的存储单元。可寻址单元数量 × 单元大小 = 容量。
- **传输单位**：对于内存，由数据总线的带宽限制，不需要等于一个字或寻址单元；对于外存，通常用块的形式。
- **访问方式**
  - **顺序存取**：按顺序从头到尾存取。存取时间依赖于数据的位置和当前位置的距离。典型的例子是**磁带**。
  - **直接存取**：每一个块都有唯一的地址，存取时先直接访问邻近区域，再顺序搜索。存取时间依赖于当前的位置和数据所在位置的距离。典型的例子是**磁盘**。
  - **随机存取**：每一个存储单元都有唯一的地址，存取时间不依赖于数据位置和之前的访问顺序。典型的例子是**随机存取存储器** (Random Access Memory, **RAM**) ，也叫**主存**。
  - **相联存取**：是随机存取的一种，通过存储内容而非物理地址进行存取。先比较字的部分内容，若命中则继续存取。存取时间不依赖于数据位置和之前的访问顺序。典型的例子是 **Cache** 。
- **性能**
  - 存取时间：对于随机存取，指**将存储地址发给存储器**到**完成传输**所用的时间；对于非随机存取，指**将读写头定位到数据所在位置**的时间。
  - 存储周期：主要针对 RAM ，包括存取时间和下一次存取开始前需要的额外时间。
  - 传输速率：对于 RAM ，等于存储周期的倒数；对于非 RAM ，有以下等式：

符号定义：

n = 位数

T_n = 读写 n 位的平均时间

T_A = 平均访问时间

R = 传输速率，单位为位/秒 (bps)

$$T_n = T_A + \frac{n}{R}$$

- **物理类型**
  - 半导体 (semiconductor) : RAM
  - 磁介质 (magnetic) ： 磁盘、磁带
  - 光学材料 (optical) : CD 、DVD
- **物理特性**
  - 易失性 (volatility) ： 断电时信息是否丢失。硬盘非易失，内存易失。
  - 可擦除性 (erasable) ： 不可擦除的存储器无法被更改，这种存储器被称为**只读存储器** (Read-Only Memory, **ROM**) 。

存储器层次结构

典型的存储器层次结构包含三层:

- 板上存储器：包括**寄存器**、 **Cache** 和**主存**，速度最快但容量最小，成本最高。
- 外部存储器：包括**磁盘**、 **CD-ROM** 等，速度较慢但容量大，成本较低。
- 离线存储器：包括**磁带**等，通常用于备份和归档。

4.2 Cache 原理

Cache 上保存的是内存的一小部分数据。CPU 取数据时， 会生成一个**读地址** (read address), 根据这个地址来分析数据所在的块是否在 Cache 中, 如果在就直接读取 Cache , 否则读取内存并更新 Cache , 同时把数据传给 CPU 。

Cache 和内存通过**块** (Block) 来交换数据。内存中每行是一个字，连续多行组织成一个块。如果内存包含 2^n 个可寻址字，则每个字有唯一的 n 位地址。为避免与内存块混淆， Cache 的基本单位使用术语“行”而非“块”。 Cache 行的大小等于内存块的大小。

4.3 Cache 设计要素

地址

虚拟内存是一种允许程序从逻辑角度寻址的技术，不需要从物理上考虑内存的容量。使用虚拟内存技术时，机器指令中的地址域包含的是虚拟地址，需要**内存管理单元** (Memory Management Unit, **MMU**) 将其翻译为物理地址。而 Cache 的位置决定了 Cache 的类型:

- **逻辑 Cache**: 位于 CPU 和 MMU 之间, 也叫虚拟 Cache 。CPU 将请求数据的逻辑地址直接发给 Cache , 如果未命中再通过 MMU 转换为物理地址，到内存中取数据并更新 Cache 。
- **物理 Cache**: 位于 MMU 和内存之间。物理 Cache 使用内存的物理地址保存数据，需要先把逻辑地址翻译成物理地址，再访问 Cache 。

容量

Cache 越大，成本越高，且用于寻址该缓存的门越多，这导致较大的 Cache 往往慢于较小的。同时，可用的芯片和电路板的面积也限制了 Cache 的大小。

三种映射方式

由于 Cache 行比内存块少，需要映射函数把内存块映射到 Cache 行上。看一个例子：

映射参数：

- **Cache 数据**：容量为 64KB，行大小为 4B，共 16K (2^{14}) 行。
- **内存数据**：容量为 16MB (2^{24} B)，需要 24 位地址寻址，块大小为 4B，共 4M 块。

- **直接映射**

把每个内存块映射到**唯一**的 Cache 行。由于块大小为 4B，需要低 2 位来确定块中的字，剩下的高 22 位为内存中的块地址标识。由于 Cache 有 2^{14} 行，需要 14 位行号，则剩下 8 位用于标记是否命中。内存地址的划分如下：

Tag 标签	Line 行号	Word 字
8 位	14 位	2 位

直接映射实现起来简单便宜，缺点是**命中率低**，替换操作频繁。如果需要访问的两个不同块映射到同一行，这两个块就会不断地被交换进 Cache，从而降低命中率。这种现象被称为**抖动** (thrashing)。降低代价的方法是把被丢弃的行保存下来，以防再次被需要，实现这种回收的技术是**受害者缓存** (victim Cache)。受害者缓存是一种全相联映射 Cache，大小一般为 4 ~ 16 个 Cache 行，位于 L1 Cache 和下一级存储层级之间。

- **全相联映射**

内存块可以映射到**任意 Cache 行**，使用标签位唯一确定内存块。内存地址的划分如下：

Tag 标签	Word 字
22 位	2 位

全相联映射的灵活性最高，也最复杂。由于标签位很长，需要复杂的电路来并行比较标签。当新的块读入 Cache 时，全相联映射可以灵活地决定替换哪个块，使用替换算法可以提高命中率。

- **组相联映射**

Cache 由一些**组** (set) 构成，每组包含若干**连续行**，内存块可以映射到固定组的任意行。**k 路组相联**表示一组包含连续 k 行。以 2 路组相联为例，同样需要 2 位来确定块中的字，由于 Cache 有 2^{14} 行，可以分为 2^{13} 组，则需要 13 位组号，剩下 9 位用于标签。内存地址划分如下：

Tag 标签	Set 组号	Word 字
9 位	13 位	2 位

这个例子中，考虑内存地址为 123456H 的块，取中间 13 位得到组号 0 1101 0001 0101 = 0D15H。同组的内存地址组号相同，只有 Tag 变化，也就是从 **0000 0000 0** 011 0100 0101 0100 到 **1111 1111 1** 011 0100 0101 0100。

替换算法

不同映射方式采用不同替换策略。

- 对于直接映射，内存块对应固定的 Cache 行，没有其他选择。
- 对于全相联和组相联映射，有 4 种常见的替换算法，均由硬件实现：
  - **最近最少使用** (Least Recently Used, **LRU**)：替换最久没有被引用的块
  - **先进先出** (First In First Out, **FIFO**)：替换 Cache 中留存最久的块
  - **最不常使用** (Least Frequently Used, **LFU**)：替换引用次数最少的块
  - **随机替换** (Random)：随机选择一行替换

写策略

- **写直达** (write through，也译作写穿透/通写)：写 Cache 的同时写内存，以保证内存里的数据永远是新的，替换时可以直接扔掉旧数据。缺点是会带来大量的内存流量，减慢速度。
- **写回** (write back)：只写 Cache，不写内存。当 Cache 发生更新时，设置一个更新位，也叫**脏位** (dirty bit)。当一个块被替换时，只有脏位被置 1 时，将该块写回内存。缺点是会导致部分内存是无效的， I/O 模块只能通过 Cache 访问。

Cache 一致性

多 CPU 共享内存时，由于每个 CPU 有自己的 Cache，会存在一致性 (coherency) 问题。若某个 CPU 的 Cache 做了修改并且没有回写到内存，那么内存中的数据无效，其他 Cache 上该数据也无效。解决一致性问题的方法如下：

- **写直达的总线检测** (bus watching)：每个 Cache 控制器监视地址线，如果其他的处理器写内存，就设置相应的 Cache 行为无效。
- **硬件透明** (hardware transparency)：用硬件方法确保所有通过 Cache 对内存的更新同时反映到所有 Cache 中。
- **非 Cache 存储器** (noncacheable memory)：部分内存内容由多个 CPU 共享，这部分就是非 Cache 的。所有对共享内存的访问都是 Cache 未命中，因为这部分不会复制到 Cache 中。

数据行大小

数据块被放入 Cache 中时，除了所需数据字，一些相邻的字也被放入。由于局部性原理，数据块越大，命中率先升高，因为相邻字也有可能被引用；若块继续变大，命中率会下降，因为：

- 块越大， Cache 能容纳的块数越少，数据很快会被覆盖。
- 块越大，附加的数据字离所需字越远，不满足局部性，不容易被需要。

块大小没有确定的最优值，一般大小在 8 ~ 64B 接近最优。对于 HPC 系统， 64B 和 128B 比较常用。

Cache 数量

- **多级 Cache**

随着逻辑密度增加， Cache 可以和 CPU 放在一个芯片上，这减少了访问外部总线，提高了性能，但是通常还是需要片外 Cache。最简单的**多级 Cache** 包括两级：片上的 L1 和片外的

L2 。其中 L2 使用 SRAM ，速度快于 DRAM 和 ROM ，且 L2 使用**单独的数据通道**，减轻了系统总线负担。现在许多 CPU 将 L2 集成到片上，并添加了 L3 缓存，以提高性能。

- **混合式与分离式 Cache**
  - 混合式 (unified)：数据和指令缓存到同一个 Cache 。好处是设计简单，容易实现，能自动平衡指令和数据的获取，命中率高。
  - 分离式 (split)：将 Cache 分为两部分，分别存储指令和数据，通常使用两个 L1 实现。好处是消除了取指、译码和执行对 Cache 的**竞争**使用，对**流水线**设计非常重要。

7. 输入 / 输出

7.1 外部设备分类

- **人机交互设备**: 适用于计算机用户与计算机之间的交流。例如: 显示器、键盘、鼠标、打印机。
- **机器交互设备**: 适用于计算机与电子设备之间的交流。例如: 磁盘驱动器、磁带机、传感器、控制器。
- **通信设备**: 适用于与远程设备进行数据交换。例如: 网卡、调制解调器。

7.2 I/O 模块

I/O 模块的功能:

- **控制和定时** (Control and Timing): 协调内部资源（如 CPU 、内存）与外部设备之间的数据流。
- **处理器通信** (Processor Communication): 处理器和外设之间的通信，包括:
  1. 命令解码: 处理器发指令给 I/O 模块， I/O 模块对指令解码，确定需要完成的操作。
  2. 数据传输: 数据传送是双向的，通过数据总线来完成。
  3. 状态报告: 向 CPU 报告状态，以决定当前的 I/O 操作是否进行，以及报告各种错误信息。
  4. 地址识别: 每个外设都有一个地址，类似内存单元。处理器的操作指令包括外设地址，I/O 模块根据地址，确定对哪个外设进行操作。
- **设备通信** (Device Communication): 向外部设备发送**命令**、接收**状态**信息以及交换**数据**。
- **数据缓冲** (Data Buffering): 解决处理器（高速）与外部设备（低速）之间传输速率不匹配的问题。
- **错误检测** (Error Detection): 检测并报告由外部设备故障或传输干扰引起的错误。

I/O 模块的结构:

- 数据寄存器: 到 I/O 设备的数据缓存
- 状态/控制寄存器: 保存当前状态信息或接收来自处理器的控制信息，和数据总线相连
- 总线接口: 包括数据总线、地址总线和控制总线
- I/O 逻辑: 接收处理器发送的操作命令，接收 I/O 地址信息，控制设备进行操作
- 外部设备接口: 和外设进行交互

三种 I/O 技术

- 编程式 I/O (Programmed): 处理器执行程序来控制 I/O 操作
- 中断式 I/O (Interrupt-driven): 处理器发送 I/O 命令，然后执行指令，直到 I/O 准备好并向处理器发送中断
- DMA (Direct Memory Access): 处理器发送指令给 DMA 控制器，数据传输过程无需处理器参与

  接下来会详细阐述这三种技术。

7.3 编程式 I/O

编程式 I/O 中， CPU 对 I/O 有直接控制权，这包括感知设备状态、发送读写命令和传输数据。

编程式 I/O 的过程

CPU 遇到 I/O 相关指令时，向 I/O 模块发送请求命令， I/O 模块执行请求的操作并设置状态。 I/O 不会通知或中断 CPU ，需要 CPU 定期检查 I/O 模块状态，直到发现操作完成。这需要 CPU 等待，会降低效率。

I/O 命令 (Command)

执行 I/O 相关指令时， CPU 会发送一条地址和一条 I/O 命令，地址用于指定 I/O 模块和设备，命令包括四类：

- 控制：告诉 I/O 模块做什么
- 测试：检测 I/O 模块状态
- 读：使 I/O 模块从外设获取数据，并将其放入模块内部的数据寄存器
- 写：使 I/O 模块从数据总线获取数据，然后传递给外设

I/O 指令 (Instruction)

I/O 指令是 **CPU 从内存取出**的指令， I/O 命令是 **CPU 向 I/O 模块发送**的命令。两者通常是一一对应的。一般会有多个设备连接到系统，每个设备有唯一的地址， CPU 发送的 I/O 命令中包含了所需设备的地址。有两种寻址模式：

- **内存映射 I/O** (Memory-mapped I/O)

整个地址空间被分给内存和 I/O ，两者统一编址，使用相同的指令访问它们，不需要额外的信号和指令。 I/O 的状态和数据寄存器也会被当作内存位置。

- **独立 I/O** (Isolated I/O)

I/O 使用**独立地址空间**，总线中需要**选择线**来指定地址指向 I/O 或内存。对于 I/O 需要特殊的命令才能访问。

7.4 中断式 I/O

中断式 I/O 中，CPU 无需等待和检查 I/O 状态，而是继续执行指令。I/O 处理完成后，会发送一个中断给 CPU 。CPU 会在每个指令结束后检测中断，如果有中断则保存当前指令上下文，接着处理中断。处理完成后恢复上下文，继续执行指令。

设计问题

中断式 I/O 存在的设计问题：有多个 I/O 模块时，需要确定发出中断的那个模块；且需要处理多个中断。

- 对于前者，常用的技术如下：
  - **多个中断请求线**，每个设备有独立的中断请求
  - **软件轮询**：中断服务程序轮询每个 I/O 模块，以确定哪个发出中断。一般使用 `TEST I/O` 命令或查询 I/O 模块的**状态寄存器**。缺点是**耗时**。

- **硬件轮询**：所有模块共享中断请求线， CPU 发现中断时发送一个中断响应信号，这个信号通过 I/O 模块传播，直到到达请求的模块，请求模块通过设置一个**向量** (vector) 来响应， CPU 用这个向量作为指针指向设备服务程序。这种技术被称为**向量中断**。
- **总线仲裁**：也是一种**向量中断**。I/O 模块首先获得**总线控制权**，然后才能触发中断请求线。一次只有一个模块能发出这个请求。当 CPU 检测到中断时，它会在中断响应线上做出响应，然后请求模块将其向量放置在数据线上。

- 对于后者，有如下方法：
  - **先进先出** (FIFO)： 没有优先级，先发生先处理。
  - **多优先级中断**：使用多个中断线，每个都有一个优先级。对于**硬件或软件轮询**，由**轮询顺序**决定优先级；对于**总线仲裁**，由**仲裁方式**决定优先级。

7.5 直接存储器访问

前两种方式都需要 CPU 干预，而**直接存储器访问** (Direct Cache Access, **DMA**) 可以在内存和 I/O 之间直接传输数据。DMA 模块是总线上附加的硬件模块，需要系统总线传输数据，可以代替 CPU 完成 I/O 操作。DMA 只能在处理器不需要总线的时候使用总线，或者强迫 CPU 临时挂起，后者被称为**周期窃取**。

DMA 操作过程

CPU 向 DMA 模块发送命令来告诉 DMA 如下信息：

- 读写请求，使用 CPU 和 DMA 之间的读写控制线。
- I/O 设备地址，通过数据线传送。
- 内存块读写的起始位置，通过数据线传送，存储在 DMA 的地址寄存器中。
- 需要读写的数据量，通过数据线传送，存储在 DMA 的数据计数器中。

接着， CPU、DMA、I/O 设备交换握手信号， DMA 把整个数据块在 I/O 设备和内存之间传输，传输结束后发送一个中断信号给 CPU 。CPU 只在传输开始和结束时参与。

周期窃取 (cycle-stealing)

周期窃取是指 DMA 模块准备好一个字的数据后，接管总线一个周期，该周期内传输完一个字后交还总线控制权。周期窃取**不是中断**，不需要 CPU 保存上下文。 CPU 在访问总线之前挂起，也就是说，会在存取操作数之前挂起。这会减慢 CPU 速度，但提高了 CPU 的利用率。

DMA 配置方式：

- **单总线独立 DMA**： 所有模块**共享相同的系统总线**， DMA 是 CPU 的代理，用**编程式 I/O** 在内存和 I/O 之间传输数据。每次传送消耗两个总线时钟周期，分别用于和 I/O 以及内存交换数据。
- **单总线集成 DMA-I/O**： I/O 模块通过 DMA 和总线互连， DMA 和 I/O 是一对一或者一对多的关系。这种方式减少了系统总线开销，因为 DMA 和 I/O 不需要总线交换数据。
- **I/O 总线**：在 DMA 和 I/O 设备之间配置一条 I/O 总线，以减少 DMA 模块的接口数量。只有 DMA 与内存交换数据的时候会使用系统总线。

7.6 直接高速缓存访问（第 10 版新增）

网络通信设备 I/O 速率的急剧增加，使 DMA 难以满足增长的需要，因此通过**直接高速缓存访问** (Direct Cache Access, **DCA**) 来提高性能。Intel 提出了 DDIO (Direct Data I/O) 技术来实现 DCA 。

DDIO 方案

外部网卡和 CPU 通过**最后一级 Cache** (Last Level Cache, **LLC**) 直接交换数据，绕过速度较慢的内存。这需要把网络报文存储在 LLC 中，需要其容量足够大。下面讨论 DDIO 和 DMA 在包输入和包输出过程的区别。

包输入

- 对于 DMA :
  1. I/O 控制器准备写内存。
  2. 设置待写入的内存地址对应的 Cache 行无效，以保持 Cache 和内存一致性。
  3. 通过 DMA 将包写入内存。
  4. CPU 通过 Cache 从内存中读取新写入的数据。
- 对于 DDIO :
  1. I/O 控制器给输入的数据包分配一个内存地址。
  2. 如果 Cache 命中，也就是说已经存在该地址的数据，则直接更新 Cache ，把新数据覆盖到对应的 Cache 行，而不同步更新内存。
  3. 如果 Cache 不命中，也就是说该地址不在 Cache 中，则在 LLC 中分配一块新的 Cache 行给数据包，而不写回内存。
  4. CPU 根据内存地址访问包。

包输出

- 对于 DMA :
  1. TCP/IP 协议处理程序从应用程序缓冲区读取数据到 DMA 缓冲区。
  2. Cache 未命中，使数据从内存被读取到 L3 Cache 。
  3. I/O 控制器从 L3 Cache 中读出数据并传输。
  4. 数据从 Cache 中清除并写回到内存。
- 对于 DDIO :
  1. TCP/IP 协议处理程序创建要传输的包，存储到 L3 Cache 的分配空间中。
  2. I/O 控制器通过总线读取，读操作的数据由 Cache 提供，不会访问内存。

7.7 I/O 通道和处理器

随着 I/O 功能的演进， I/O 模块增强为独立的处理器，有专门的指令集， CPU 指示 I/O 处理器执行 I/O 程序，此时 I/O 模块被称为 **I/O 通道**。再后来， I/O 模块有了自己的存储器，可以控制大量 I/O 设备，把 CPU 的参与度降到最低，此时 I/O 模块称为 **I/O 处理器**。这里两者通称 I/O 通道。常见的 I/O 通道有两种，分别为**选择通道**和**多路复用通道**。前者控制多个高速设备，代替 CPU 对 I/O 模块进行控制；后者同时处理多个设备的传输，将数据交错穿插在一起。

9. 计算机算术

9.1 算术逻辑单元

算术逻辑单元 (Arithmetic Logic Unit, ALU) 本质是组合逻辑电路，用于完成算术和逻辑计算。CPU 的**控制单元**告诉 ALU 做什么操作，并控制数据的输入输出；**寄存器**将数据送给 ALU 进行运算，结果保存在寄存器中。计算完成后 ALU 会设置一些**标志**，同样保存在寄存器中。

9.2 整数的表示

无符号表示

这种表示法用于非负整数。n 位无符号数的表示范围为 $0 \sim 2^n - 1$

原码表示（符号-幅值）

最高位为符号位，0 表示正数，1 表示负数。不难看出，n 位原码表示法的表示范围为 $-\left(2^{n-1}-1\right) \sim 2^{n-1}-1$。然而，这种表示法下算术计算需要考虑符号和幅值，且 0 有两种表示：1000 0000 和 0000 0000，也就是 $+0$ 和 -0

反码表示

正数和无符号数相同，负数为其相反数的按位取反。例如 3 表示为 0011，则 -3 表示为 1100。这种表示法下的 0 依然有两种表示：1111 1111 和 0000 0000。范围与原码相同。

补码表示

负数是其相反数的按位取反再加 1。例如：5 表示为 0101，则 -5 表示为 1011。计算方法为：

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

不难得到 $(1011)_2 = -2^{4-1} + 2^0 + 2^1 = -5$。这种表示法的优点是算术计算更容易，且 0 只有一种表示：0000 0000。n 位补码表示法的表示范围为 $-2^{n-1} \sim 2^{n-1}-1$。注意这个范围不是对称的。

移码表示

n 位移码表示法的偏移量为 $2^{n-1}-1$，表示范围为 $-\left(2^{n-1}-1\right) \sim 2^{n-1}$。计算方法为存储值（无符号二进制数）减偏移量得到实际值。以 8 位二进制数为例，偏移量为 $2^{8-1}-1=127$，那么存储值 0000 0000 表示的实际值为 -127，存储值 1111 1111 表示的实际值为 $+128$。

符号位扩展

对于原码，将符号位移动到扩展后的最高位，剩下的位补 0 即可。例如 1011 扩展后为 1000 0011。

对于补码，正数补 0，负数补 1。也就是说，最高位是什么就补什么。例如 1011 扩展后为 1111 1011。

9.3 整数的运算

取反

原码：只需要对符号位取反。

补码：按位取反加一。例如：

$+18 = 0001\ 0010$ 取反后得到 $1110\ 1101$，再加 1 得到 $1110\ 1110 = -18$。

加法

二进制加法，注意检查**溢出**。不同的表示法判断溢出的方法不一样。异号相加或同号相减都不会溢出。对于补码表示法，同号相加时若结果符号位相反，则一定溢出。

减法

$a - b = a + (-b)$，也就是说，先取反得到减数的补码，再加到被减数。其本质是取反和加法的组合，所以判断溢出方法和加法相同。

乘法

二进制乘法类似于十进制，两个 n 位二进制数相乘，结果最多为 $2n$ 位。下面展示两个无符号数相乘的计算过程：

```
        1 0 1 1      被乘数 11
      × 1 1 0 1      乘数   13
-------------------
        1 0 1 1      部分积
      0 0 0 0
    1 0 1 1          如果乘数位是 0，结果为 0。
  1 0 1 1            如果乘数位是 1，结果为被乘数。
-------------------
1 0 0 0 1 1 1 1      结果 143 (需要双倍的比特位)
```

对于补码表示的有符号数，这种计算方法并不适用。以 $3 \times (-4)$ 为例，$0011 \times 1100 = 00100100 = (32)_{10}$，显然不对。正确的做法是转成正数乘法，根据结果的符号决定是否取反。

除法

除法较为复杂，尤其是对有负数的情况。课程对此不做要求。

9.4 浮点数的表示

科学计数法

十进制中常用科学计数法表示大数字，例如 6.02×10^{23}，其中 6.02 是**尾数** (significand)，10 是**基数** (radix)，24 是**指数** (exponent)。二进制的科学计数法与十进制类似，只是基数是 2。下图展示了 32 位浮点数的组成：

$$\underbrace{0}_{\text{sign (1 位)}}\ \underbrace{0000\ 0000}_{\text{exponent (8 位)}}\ \underbrace{0000\ 0000\ 0000\ 0000\ 0000\ 000}_{\text{significand (23 位)}}$$

其中，指数部分用**移码**表示，小数点在有效值（也就是尾数）最高位的右边第一位。对于单精度浮点数，移码有 8 位，则偏移量为 $2^{8-1}-1=127$。实际值加上偏移量得到存储值。

看一个例子：$-1.1010001\times 2^{10100}$

- 因为是负数，**符号位**为 1
- **指数**部分为 $10100+01111\ 1111\big(\text{偏移量}\big)=1001\ 0011$
- **尾数**部分为 101 0001 0000 0000……0000
- 拼接起来得到完整的二进制表示：1 10010011 10100010000……0000

注意，小数点左边的 1 不需要表示，这是因为这个数已经被**规格化** (normalize) 了。规格化后的数最高位为 1，只需要隐含即可。

浮点数的范围和精度

影响浮点数范围的因素是**尾数**和**指数**。以 32 位单精度浮点数为例，指数的范围 $-(2^7-1)\sim 2^7$，最大的尾数为 $1.\underbrace{111\cdots\cdots}_{23\text{ 个 }1}=2-2^{-23}$。可以得到：

正数的表示范围：$2^{-127}\sim\big(2-2^{-23}\big)\times 2^{128}$

负数的表示范围：$-\big(2-2^{-23}\big)\times 2^{128}\sim -2^{-127}$

超出浮点数范围会导致溢出。绝对值太大是**上溢出**，绝对值太小（太接近 0）是**下溢出**。浮点数的**精度**体现在尾数最低位，比如上述例子中 $2^{-23}\approx 1.2\times 10^{-7}$。

浮点数在空间上是**不均匀的**，随着绝对值越来越大，数之间的距离也越来越大，表现在数轴上就是越靠近原点越密集，越远离原点越稀疏。

IEEE 754 标准

这个标准规定了浮点数的格式，包括最常用的 32 位单精度和 64 位双精度。下图展示了两种精度的格式：

$$\underbrace{0}_{\text{sign}}\ \underbrace{0000\ 0000}_{\text{exponent (8 位)}}\ \underbrace{0000\ 0000\ 0000\ 0000\ 0000\ 000}_{\text{significand (23 位)}}$$

$$\underbrace{0}_{\text{sign}}\ \underbrace{000\ 0000\ 0000}_{\text{exponent (11 位)}}\ \underbrace{0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000}_{\text{significand (52 位)}}$$

注意：IEEE 754 标准中，**指数的极值**表示特殊情况：

- **指数全为 1**
  - 尾数全为 0 表示**无穷**，无穷的正负取决于符号位
  - 尾数不为 0 表示 **NaN** (Not a Number)
- **指数全为 0**
  - 尾数全为 0 表示 **0**，由符号位决定 $+0$ 还是 -0
  - 尾数不为 0 表示**非规格化的值**，隐含的小数点左边一位为 0。

因此，真正的指数只有 2046 个 $(-1022\sim 1023)$，正数范围为 $2^{-1022}\sim\big(2-2^{-52}\times 2^{1023}\big)$。

9.5 浮点数的运算

加减法

从一个例子来看浮点数加减法的步骤： $1.011 \times 2^5 + 1.001 \times 2^3$

1. **检查 0**：两数存在 0 则另一个数就是结果。这里显然没有 0 。
2. **对齐尾数**： $1.001 \times 2^3 = 0.0101 \times 2^5$
3. **加减尾数**： $1.011 + 0.0101 = 1.1011$
4. **规格化**： 1.1011×2^5

乘除法

同样的例子来看乘除法的步骤： $(1.011 \times 2^5) \times (1.001 \times 2^3)$

1. **检查 0**：两数存在 0 则结果为 0 。这里显然没有 0 。
2. **加减指数**：由于指数都是**移码**表示，两个指数相加则结果中有两个偏移量，需要减去一个。相减则加一个。需要考虑溢出。这里直接计算实际值，不用处理偏移量：$5 + 3 = 8$
3. **乘除尾数**：需要考虑符号。$1.011 \times 1.001 = 1.100011$
4. **规格化**：1.100011×2^8
5. **四舍五入**：$1.100011 \times 2^8 \approx 1.1002^8$

10.指令集：特性与功能

10.1 机器指令特征

指令表示

计算机指令的基本要素包括**操作码**、**源操作数**和**目的操作数地址**以及**下一条指令地址**。其中下一条指令地址通常是隐含的。简单的指令格式如下图：

Opcode	Operand Reference	Operand Reference
4 bits	6 bits	6 bits

指令类型

1. **数据处理**：算术和逻辑指令
2. **数据存储**：数据在内存和 CPU 寄存器之间的传送
3. **数据传送**：数据在 CPU 和 I/O 之间的传送
4. **程序流控制**
    - CPU 执行控制的指令
    - 测试 (Test)：用于测试字的值或者计算的状态
    - 分支 (Branch)：用于分支到不同指令集合

地址数量

- 3 个地址：操作数 1, 操作数 2 , 结果
- 2 个地址：其中 1 个地址为源操作数和结果的复用, 即 $a = a + b$
- 1 个地址：隐含了第 2 个操作数地址
- 0 个地址：所有的地址都是隐含的，通常用栈来处理操作数

指令集设计要素

1. **操作指令表**：决定计算机能执行哪些操作，以及操作的种类和复杂程度。
2. **数据类型**: 操作数类型
3. **指令格式**：操作码长度、地址数量、各个字段的大小等
4. **寄存器**：指令可引用的 CPU 寄存器数量及其用途
5. **寻址方式**：指定操作数地址的方式。 11 章中有详细阐述。

10.2 操作数类型

数据都是以二进制串的形式保存的。指令中确定了操作数据的类型，因此，数据的类型取决于指令的类型。

- **地址**：无符号整数, 用于进行实际操作数的寻址
- **数值**：二进制整数、二进制浮点数、十进制数
- **字符型**: ASCII 码
- **逻辑数据**: 位, 标志 (flag)

10.3 操作类型

算术运算 (Arithmetic)

算数运算主要由 CPU 完成，实际在 **ALU (算术逻辑单元)** 中执行。算术运算可能包含数据传送操作。计算结束后会设置**状态码**或**标志位**，用以表示计算结果（是否溢出、出错）。

单操作数指令包括：绝对值、取反、加/减 1 。

双操作数指令包括：加、减、乘、除。

操作数包括：有符号数或定点数、浮点数、压缩的十进制数

逻辑运算 (Logical)

基本的逻辑操作包括：与、或、非、异或。扩展的逻辑操作包括：

- **测试**：进行特定条件的测试，并设置标志位
- **比较**：对两个或多个操作数进行比较，并设置标志位
- **设置控制变量**：用于设置控制位的指令
- **移位**：左移/右移
  - 逻辑移位：左右移都补 0，不考虑最高符号位
  - 算术移位：右移补符号位，左移保留符号位
- **旋转**：循环移位

数据传送 (Data Transfer)

数据传送对处理器来说是最基本、最简单的操作, 实现了数据从一个位置到另一个位置的移动。常见的数据传送指令包括：移动、存储、加载、交换、清除、设置、进栈、出栈等。

需要明确：

- 源操作数和目标操作数的位置，可能在内存、寄存器或栈顶。
- 传送数据的长度
- 寻址模式

转换 (Conversion)

用来改变数据格式或对数据格式进行操作。比如将二进制转换为十进制, 或从压缩的十进制转换为二进制。

输入输出 (I/O)

包括：读、写、启动、测试。在第 7 章中有详细阐述，可以采用多种方式：独立 I/O、内存映射 I/O、DMA、I/O 处理器。

系统控制 (System Control)

系统控制指令是特权指令，需要 CPU 处于内核状态 (kernel mode)，通常保留给操作系统使用。包括：

- 读写控制寄存器
- 读写存储保护锁
- 在多程序系统中访问进程控制块

控制转移 (Transfer of Control)

控制转移指令根据当前指令的执行结果, 决定下一步需要执行的指令, 也就是改变了原有指令执行顺序。常见类型:

- **分支 (Branch)**: 也就是 jump 跳转，把下一个指令的地址作为当前指令的操作数，通常是条件分支指令。
- **跳过 (Skip)**: 跳过下一个指令的执行, 包含一个隐含的地址, 不需要指明下一个指令的地址。
- **过程调用 (Subroutine call)**: 调用一段子程序，由调用指令调用，由返回指令返回，且调用和返回需要匹配。CPU 需要保存返回地址，还需要传递参数（通过寄存器、内存单元或栈）。为过程调动而存储的栈信息称为栈帧。返回的地址会被保存在**寄存器**、**过程起始地址**或**栈顶**。

10.4 端序支持

大端端序 (Big Endian): 将高位地址存进低数字位，即从左往右。IBM370，Motorola 68000，RISC，互联网是大端序。

小端端序 (Little Endian): 将地位地址存进高数字位，即从右往左。奔腾，VAX，Alpha 采用小端序。

ARM 支持两种端序，由系统寄存器中的 E-bit 控制。E-bit = 0 为大端序，E-bit = 1 为小端序。

11. 指令集：寻址模式与格式

11.1 寻址

什么是寻址模式

指令元素包括：**操作码**，**源操作数**，**目的操作数**，**下一指令地址**。

操作数可能的位置：**立即数**、**寄存器**、**内存**、**I/O**。

寻址模式确定如何获取指令中的操作数。

为什么需要寻址模式

如果操作数在内存中，其地址字段不能太长，而我们希望访问更大的内存空间，这需要寻址模式。

寻址模式需要权衡：**地址范围**和**寻址灵活性**、**内存引用数量**和**地址计算复杂度**。

优点：

- 扩充了可寻址的**地址空间**
- 提高了寻址的**灵活性**
- 提供更好的程序架构，帮助设计更灵活的程序

常用寻址模式

符号定义：

A = 指令中地址字段的内容

R = 指令中地址字段的内容，指向一个寄存器的

EA = 有效地址（实际地址），包含被引用操作数

(X) = 内存位置 X 或寄存器 X 的内容

1. **立即数 (immediate)** 操作数 $= A$

   操作数 = 地址域，不需要去内存取数，但是地址域长度有限，因此值域有限。

   `ADD 5`：把 5 加到累加器中，5 就是操作数。

2. **直接寻址** $EA = A$

   指令中给出了操作数在主存储器中的地址，**有效地址 = 地址域**，通过一次存储器访问取数，不需要额外的有效地址的计算。地址长度有限，因此寻址空间有限。

   `ADD A`：按照地址 A 到内存中取操作数，将其加到累加器中。

3. **间接寻址** $EA = (A)$

   地址域指向的内存单元包含操作数的地址。需要 **2 次访问存储器**，可以嵌套。地址空间更大。若存储器字长为 n，地址空间为 2^n。

   `ADD (A)`：与直接寻址不同的是，按照地址 A 取到的不是操作数，而是操作数的地址，需要根据此地址再次访问内存，才能得到操作数。

4. **寄存器寻址** $EA = R$

将操作数保存到地址域指定的寄存器中，**不需要访问内存**，取值快。寄存器数量有限，地址域很小。

5. **寄存器间接寻址** $EA = (R)$

   寄存器 R 中的值是操作数在存储器中的地址。类似于间接寻址，但寄存器的访问时间很短，主要时间开销在于访问存储器。比间接寻址快。

   - 间接寻址：**2 次**内存访问
   - 寄存器间接寻址：**1 次**内存访问 + **1 次**寄存器访问

   寄存器的字长为 n ，最大寻址空间为 2^n。

6. **偏移寻址** $EA = A + (R)$

   A 会被直接使用，R 会指向一个寄存器，该寄存器的内容与 A 相加后可得到有效地址。

   1. **相对寻址** $EA = A + (PC)$

      PC = Program Counter

      地址段 A 为有符号偏移量。隐式引用的寄存器是程序计数器 (PC)，因此也被称为 PC 相对寻址。相对寻址利用了**局部性原理**，目标地址通常在当前指令附近，在 cache 中的概率大，因此存取速度快。

   2. **基址寄存器寻址** $EA = A + (BR)$

      BR = Base Register

      寄存器 BR 存储主存地址，地址段包含一个相对于该地址的偏移量。寻址过程包括：访问 1 次寄存器、进行 1 次加法运算、访问 1 次主存。

   3. **变址寻址** $EA = A + (R)$

      地址段 A 指向主存，寄存器 R 包含偏移量，这与基址寄存器寻址相反，而包含的位数比后者更多。

      自动变址：$EA = A + (R), (R) \leftarrow (R) + 1$

      自动变址用于迭代任务，变址与指令周期同步进行。

   4. **混合寻址** (间接寻址 + 寄存器间接寻址)

      - **后变址**： $EA = (A) + (R)$ 间接寻址之后再变址。

        取出 A 指向的内存单元中的地址，经过变址后得到操作数的实际地址，从而得到操作数。

      - **前变址**： $EA = (A + (R))$ 变址之后再间接寻址。

        直接对地址段 A 进行变址，得到操作数间接地址 $A + (R)$，访问内存得到操作数直接地址，再访问一次得到操作数。

7. **栈寻址 stack**

   操作数隐含在**栈顶**。

   `ADD`：弹出栈顶的两个数，相加后将结果压入栈中。

11.2 指令格式

指令格式的关键要素

1. **操作码宽度**：决定了有多少种操作。操作码数量越多，指令集功能越多。
2. **操作数宽度**：影响指令长度。操作数占指令长度较大，操作数的数量、寻址模式、寻址空间的大小都会影响指令长度。
3. **寻址模式**：决定复杂性和指令长度。寻址模式越复杂，时间复杂度越高，但可以用较少的地址长度获得更大的寻址空间，节省指令长度。

指令长度的设计

1. **指令长度**

   操作码种类、存储空间大小、内存组织方式、总线架构、CPU 复杂度等因素共同决定指令长度，从而决定指令集的丰富和灵活程度。需要权衡指令集功能和空间大小。从硬件设计角度看，**指令长度和数据总线宽度应当一致**，或者是整数倍关系，否则取指周期中不能取到整数个指令。比如 32 位总线一次可以取 2 个 16 位指令。

2. **位的分配**

   需要考虑的因素：

   - **寻址模式数量**：有时可以隐式指定寻址模式，有时必须显示指定。一个指令可能有多种寻址方式。
   - **操作数数量**：地址越少，程序越长、越不灵活，所以一般支持 2 个操作数。
   - **寄存器与内存**：数据需要通过寄存器送入 CPU 处理。如果只有一个寄存器，不需要指定，但是使用起来非常麻烦。
   - **寄存器组的数量**：大多数现代机器都有一组寄存器，用于保存数据或者偏移寻址的地址。部分 (比如 x86) 有两组或更多专用组。
   - **地址范围**：可引用的地址范围由地址段位数决定。直接寻址范围小，所以一般用偏移寻址。
   - **地址的粒度 (granularity)**：按照字节或字来寻址。

3. **变长指令**

   不同操作码，指令长度可能不同，寻址方式更灵活，也增加了复杂度。**PDP-11** 采用了三种长度：16 位、32 位、48 位。

   - **16 位**：共有 8 种，包括零地址、单地址、双地址，操作码长度从 4 位到 16 位不等。共有 8 个寄存器，寄存器引用地址为 6 位，其中 3 位指定寄存器，3 位决定寻址方式。
   - **32 位**：共有 4 种，存储器地址占 16 位，寻址空间 $2^{\{16\}} = 64k$ ，操作码从 4 位到 10 位不等，寄存器引用同 16 位。
   - **48 位**：共有 1 种，2 个存储器地址占 32 位，操作码占 4 位，2 个寄存器引用占 12 位，每个寄存器引用同 16 位。

12. 处理器结构与功能

12.1 & 12.2 处理器与寄存器组成

处理器职能

CPU 的职能包括：从内存取指、指令解码、取操作数、处理操作数、写结果。

处理器组织

处理器包括**算术逻辑单元** (ALU), **寄存器** (Register), **控制单元** (CU), 由内部总线连接它们。

- ALU 内部除了算术逻辑运算单元外，还包括状态标志、移位器、求补器、算术逻辑。
- 寄存器是由一组存储单元构成的小型内部存储器，用于暂时保存指令和数据。
- CU 控制各个部件按照指令完成操作。

寄存器组成

寄存器位于存储层次结构的**顶层**，包括：

- **用户可见的寄存器**
  - **通用寄存器**

    对于真正意义上的通用寄存器，寄存器和操作码在指令集中是**正交**的，也就是说，操作码和寄存器是互不干涉、相互独立的，两者可以**自由组合**。而对于受限的通用寄存器，一般专用于浮点数或栈操作。

    某些情况下，通用寄存器可以用于**寻址**，比如寄存器间接寻址和偏移寻址。
  - **数据寄存器**

    只能用于保持数据，**不能用于寻址**。累加器 (Accumulator) 就是典型的数据寄存器。
  - **地址寄存器**

    用于某种具体的寻址方式，比如段指针、栈指针、变址寄存器。
  - **条件码寄存器**

    条件码也称**标志** (flag)，是处理器硬件根据操作的结果 (正负、0、溢出等) 设置的位，对用户部分可见。条件码可以被程序隐式地读取，但**不能被程序员手动修改**。条件码主要用于**条件分支指令**，程序根据这些标志位的状态来决定程序的执行流程 (跳转或继续顺序执行)。
- **控制寄存器**

  控制寄存器一般对用户不可见。程序执行有四个必须的寄存器：
  - **程序计数器 PC** (Program Counter)
  - **指令寄存器 IR** (Instruction Register)
  - **内存地址寄存器 MAR** (Memory Address Register)
  - **内存缓冲寄存器 MBR** (Memory Buffer Register)
- **状态寄存器：程序状态字**

  程序状态字 (Program Status Word, **PSW**) 是一个或一组寄存器，包含以下状态信息：
  - **条件码**：符号位，0，进位，相等，溢出

- **中断允许/禁止**
- **管理员模式/用户模式**：由操作系统使用，对用户程序不可用。某些特权指令和内存区域只能在管理员模式使用。

寄存器的设计问题

1. 通用 or 专用

   对于专用寄存器，寄存器类型可以隐含在操作码中，节省了位数，也降低了灵活性；对于通用寄存器，灵活性更高，但指令长度和复杂度也会增加。

2. 寄存器数量

   寄存器数量越多，指令中确定寄存器需要的位数越多，同时处理器成本越高。虽然寄存器越少访存越多，但寄存器越多**不一定能减少访存次数**。数量一般在 8 ~ 32 个为最佳。

3. 寄存器长度

   地址寄存器需要足够容纳**最大的地址**，数据寄存器应该能存放**大多数数据的值**。有时会用两个寄存器保存双倍长度的数据。

4. 对于控制和状态寄存器

   需要对操作系统提供支持，特定类型的控制信息对操作系统有特定用途。同时要控制信息在寄存器和内存之间的分配，需要在成本和速度间权衡。

12.3 指令周期

指令周期包括**取指周期**和**执行周期**，还可能包括**间接周期**和**中断周期**。

间接周期和中断周期

即**间接寻址**的过程。取指后，通过**译码**确定是否包含需要间接寻址的操作数，若有则进入间接周期。取操作数和存结果的过程中，都可能会存在间接周期。执行完成后检查是否有**中断**，若有则进入**中断周期**。

数据流

- **取指周期**:

PC 包含下一条指令的地址，这个地址被送入 MAR ，并放到**地址总线**上。控制单元要求读内存，其结果放到**数据总线**上，并被复制到 MBR ，然后送入 IR 。同时， PC 加 1 。 控制单元检查 IR 内容，若没有间接寻址则进入**执行周期**，若有则进入**间接周期**。

- **间接周期**:

MBR 最右 n 位被传输到 MAR ， 然后 MAR 将其放在**地址总线**上。控制单元请求读内存，内存将需要取的操作数地址放在**数据总线**上， MBR 读出地址。

- **执行周期**:

有多种形式，取决于被执行的指令。可能包括：读写内存、输入输出、寄存器传输、 ALU 操作。

- **中断周期**:

把 PC 的内容传给 MBR，并写入内存，特殊内存位置 (可能是栈指针) 从控制单元加载到 MAR 。PC 加载中断处理程序的地址，取中断程序的第一个指令。

12.4 指令流水线

流水线基本思想

将指令的执行分为多个阶段，多个指令的不同阶段可以并行处理。

预取 (Prefetch) 是指执行当期指令的时候取下一条指令，当前指令执行完成后，可以立即执行下一条。取指需要访问存储器 (先 Cache，后内存)，而执行不需要，因此可以加速指令的执行，但需要更多寄存器来保存阶段之间的数据。

预取的性能问题：

- 执行时间通常比取指时间长，而流水线速度受限于最慢的阶段。
- 如果存在条件分支指令，需要等待执行阶段，根据分支结果取实际需要的指令。

理想流水线的特点

- **重复相同操作**。不同操作要求的步骤不一样，影响流水线的运行。
- **操作对象之间相互独立**。各对象间没有依赖关系，在并行操作的时候才不会相互影响。
- 完整的操作**可以分解为若干个子操作**。每个子操作耗时相同，资源独立，不共享资源。若耗时不同会导致等待，资源共享会导致争抢。

流水线时序

为了获得更多的加速，我们使流水线有更多流水段。这里将指令分解为 6 段：

1. **取指 FI** (Fetch Instruction)
2. **译码 DI** (Decode Instruction)
3. **计算操作数 CO** (Calculate Operands)
4. **取操作数 FO** (Fetch Operands)
5. **执行指令 EI** (Execute Instruction)
6. **写操作数 WO** (Write Operands)

下图展示了该流水线的时序：

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
指令 1	FI	DI	CO	FO	EI	WO								
指令 2		FI	DI	CO	FO	EI	WO							
指令 3			FI	DI	CO	FO	EI	WO						
指令 4				FI	DI	CO	FO	EI	WO					
指令 5					FI	DI	CO	FO	EI	WO				
指令 6						FI	DI	CO	FO	EI	WO			
指令 7							FI	DI	CO	FO	EI	WO		
指令 8								FI	DI	CO	FO	EI	WO	

当存在**条件分支**指令时，流水线时序如下。**指令 3** 会跳转到**指令 15**，但需要到时刻 7 才能确定下一条指令，此时**指令 4 ~ 7** 已完成的部分作废，重新开始**指令 15** 的取指。从时刻 9 ~ 12 期间没有指令完成执行，这就是无法预测分支导致的性能损失，也就是**分支惩罚**。

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
指令 1	FI	DI	CO	FO	EI	WO								
指令 2		FI	DI	CO	FO	EI	WO							
指令 3			FI	DI	CO	FO	EI	WO						
指令 4				FI	DI	CO	FO							
指令 5					FI	DI	CO							
指令 6						FI	DI							
指令 7							FI							
指令 15								FI	DI	CO	FO	EI	WO	
指令 16									FI	DI	CO	FO	EI	WO

理论上流水线中段数越多，执行速度越快，但有几个因素阻碍了这一设计思路：

- 数据在**缓冲区间传递**的时间开销，这会延长单条指令的执行时间。当存在依赖时，对性能的影响更明显。
- 分的阶段越多，阶段间的控制越复杂，处理内存和寄存器依赖性、优化流水线所需的控制逻辑数量大量增加。
- **锁存延迟**：把信号和数据从一个阶段推到下一个阶段所需的延迟。流水线缓冲区操作需要时间，这会增加指令周期的时间。

流水线性能

指令流水线的周期时间 τ 是通过流水线把一组指令**推进一个阶段**所需的总时间，也就是上面时序图中的一列代表的时间。注意是一个阶段而非一条指令的时间，后者称为**指令周期**。

符号定义：

τ_i：流水线中第 i 段的电路延迟时间

τ_m：最大阶段延迟

k：流水线的阶段数

d：锁存器延迟

n：指令数

$T_{k,n}$：k 段流水线执行 n 条指令的总时间

S_k：流水线加速比

周期时间：

$$\tau = \max[\tau_i] + d = \tau_m + d, 1 \leq i \leq k$$

总时间：

$$T_{k,n} = [k + (n-1)] \times \tau$$

对于无流水线的处理器，其执行单条指令的周期显然为 $k\tau$ 。于是可以算出流水线的加速比：

$$S_k = \frac{T_{1,n}}{T_{k,n}} = \frac{nk\tau}{[k + (n-1)]\tau} = \frac{nk}{k + (n-1)}$$

指令流水线分析

指令流水线**不是理想流水线**。回顾理想流水线的特征：

- 重复相同操作：指令流水线中的指令不同，步骤也不同，会导致有些流水线阶段空闲。造成的时间开销就是**外部碎片**。
- 操作对象独立：指令间并不独立，需要检测和解决指令依赖关系。
- 可分解成耗时相同的子操作：不同阶段的延迟也不同，需要强迫所有的阶段用同一个时钟来控制，即使某些阶段速度很快，也需要被统一的时钟控制。造成的时间开销就是**内部碎片**。

12.5 流水线冒险

流水线暂停的原因：**资源**竞争，指令间的**数据**相关性、**控制**相关性，长延时操作。这引出了三种**流水线冒险**：**资源**、**数据**和**控制**。

- **资源冒险** (也叫结构冒险)

  多个指令需要相同资源时发生，这导致部分指令必须串行执行。解决方法是增加可用资源。

- **数据冒险**

  访问同一个操作数位置时发生，这导致流水线需要暂停。分为三种类型：

  - **真相关 (写后读 RAW)**

    一条指令修改数据，后续指令读这个位置的数。后续指令需要读修改后的数据，如果先读后写就会出错。

  - **反相关 (读后写 WAR)**

    一条指令先读数据，后续指令需要写这个位置。如果写操作先完成，就会使读到的数据出错。

  - **输出相关 (写后写 WAW)**

    两个指令要向同一个位置写数据。需要后续指令写的结果，但后续指令先执行写操作，最终保存的是前面指令写的结果。

- **控制冒险** (也叫分支冒险)

流水线对分支转移判断错误时发生，这导致进入流水线的指令随后必须被丢弃，流水线无法满载运行。

处理数据冒险

- 检查并等待寄存器的值可用
- 检查并消除依赖性 (14 章会提到的**寄存器重命名**)
- 预测所需的值，“推测性”执行并验证

处理控制冒险

1. **多指令流** (Multiple Streams)：复制流水线的初始部分，当遇到分支时，同时取两个方向（跳转和不跳转）的指令。缺点是会带来总线或寄存器竞争，以及如果遇到连续分支会失效。
2. **预取分支目标** (Prefetch Branch Target)：当识别出条件分支指令时，除了预取顺序的下一条指令外，还预取分支目标的指令并保存，以备跳转时使用。
3. **循环缓冲** (Loop Buffer)：使用一个小型的、高速的指令存储器，保存最近取出的指令。如果分支是跳转到缓冲区内 (如短循环)，则无需从内存重新取指。
4. **分支预测** (Branch Prediction)
   - **静态预测**：不依赖执行历史。包括：
     - 预测从不跳转 (predict-never-taken)：假定不跳转，总是取下一条指令。这种使用最多。
     - 预测总是跳转 (predict-always-taken)：假定会跳转，总是取目标指令。
     - 按操作码预测：有些指令跳转的可能性会更大。预测成功率为 75% 。
   - **动态预测**：依赖执行历史。包括：
     - 跳转 / 不跳转开关：用一个或多个位来反映指令的最近历史记录。
     - 分支历史表：记录了和分支转移相关的信息，包括分支指令地址、转移历史位，以及目标地址信息 (目标指令或其地址) ，可以在做出分支决定时就启动取指。
5. **延迟分支** (Delayed Branch)：编译器重排指令，将一条不依赖于分支结果的有用指令 (或空操作 NOP) 放在分支指令之后执行，利用这段时间让流水线完成分支判断，从而避免流水线停顿。

13. 精简指令集计算机

13.1 寄存器和指令集

指令集 (Instruction Set Architecture) 是 CPU 能理解的完整指令集合，规定了计算机执行的操作、操作数地址空间和操作数类型。指令集的寻址在 11 章中有详细介绍。

指令集与寄存器关系密切。指令集架构根据操作数来源可以分为两类：

- **寄存器 - 存储器**：操作数可以来自寄存器，也可以来自存储器，例如 Intel 80386 。
- **寄存器 - 寄存器**：所有操作数来自寄存器，例如 ARM , MIPS 。

早期 CPU 只有一个寄存器，必须存储器寻址。后来，寄存器数量增多，指令集和寻址方式也更灵活。寄存器有如下特征：

- 与 ALU 、控制器在同一个芯片上，距离最短
- 直接和 ALU 连接
- 是速度最快的存储部件
- 成本高，数量少
- 寻址位数短

13.2 精简指令集架构

高级语言 HLL

随着硬件成本下降，软件成本上升，更强大、更复杂的**高级语言 (High-Level Language, HLL)** 被开发出来。由于高级语言和指令集的差距越来越大，需要更复杂的编译器来翻译高级语言，程序体积也更大，且一条高级语言语句需要多个指令来完成，降低了执行效率。这导致了**语义差距** (semantic gap)。缩小语义差距的一种方案就是**复杂指令集计算机 (CISC)** ：提供更多类型的指令、更多寻址模式以及在硬件中实现的 HLL 语句，指令长度可变。

CISC 的理想效果：

- 简化编译器的编写。指令集提供的指令类型更多，编译器可以用最合适的指令翻译 HLL , 编译后的指令条数少，占用空间小。
- 提高执行效率。原本需要多条指令完成的 HLL 语句，现在只用一条指令完成。
- 支持更复杂、更精细的 HLL 。更多寻址模式可以帮助实现复杂的指令。

指令执行的特征：

- **操作类型**：决定 CPU 的功能及其与内存的交互。包括：
  - 赋值语句
  - 条件语句 (loop and if)
  - 过程的调用与返回
- **操作数**：操作数类型、使用频率决定了存储他们的**内存组织**和访问他们的**寻址模式**
  - 操作数引用的类型主要是局部标量变量
  - 优化方向应集中于局部变量的存储和访问

- **执行顺序**：决定了控制与流水线结构
  - 是最耗时的操作，取决于过程处理时参数和变量的数量，以及嵌套的深度。
  - 传递参数数量大部分小于 6 个，大部分变量是局部的。

研究表明，CISC 的理想目标与其实现方式相矛盾。我们需要优化最常用和最耗时的功能来提高效率，这引出了**精简指令集计算机 (RISC)**。

RISC 的特征：

- 使用**大量寄存器**或者使用**编译器优化**寄存器的使用

  数据运算只能在寄存器中运行，内存访问只有 3 种寻址模式。除了加载 / 保存指令，其他指令都是针对寄存器操作。

- **指令集有限且简单**

  RISC 采用规范化的、定长的指令格式，操作类型有限，只有 8 位操作码，这使取指和解码更容易。大多数操作是**寄存器 - 寄存器**的。每个机器周期一条指令，采用硬布线设计，没有微代码。

- 强调**指令流水线**的优化

  精心设计指令流水线，更好地满足条件分支和过程调用对流水线的影响，每个指令都是条件执行，以减少分支。

RISC 和 CISC 相互借鉴，随着体系结构与微电子技术的进步， CISC 的劣势缩小。 Intel 的服务器 CPU (CISC) 占据 95% 的市场份额，而 ARM (RISC) 占据了嵌入式市场的主要份额。

13.3 大寄存器组的使用

为了快速访问操作数，最小化**寄存器 - 存储器**操作，需要将访问频率最高的操作数保存到寄存器中。有两种方法：

- **软件方法**：编译器为使用最多的变量分配寄存器。这需要复杂的程序分析。
- **硬件方法**：使用更多寄存器，保存更多的变量。

大多数操作数是针对局部标量变量的，所以将其存放在寄存器中，这符合局部性原理。然而，过程（函数）的调用改变了局部性，参数需要传递、结果需要返回，被调用程序的变量需要恢复。解决方案是寄存器窗口。

寄存器窗口

将寄存器划分为多个小的寄存器组，这些寄存器组就是**寄存器窗口 (register windows)**。每个寄存器组包含三个区域：**参数寄存器**、**局部变量寄存器**、**临时变量寄存器**。其中临时变量寄存器和下一级过程的参数寄存器物理上相同，这使得参数传递无需移动数据。任何时刻只有一个窗口可见，但窗口的数量不受限制。

由于寄存器数量有限，更早的过程调用需要保存到内存中，嵌套深度减少时再将其恢复到寄存器中。这种方式被称为**环形缓冲区**。寄存器窗口以一种部分重叠的形式形成一个环形。当环形寄存器窗口都充满了后，再有过程调用，把最早的寄存器窗口保存到存储器。调用时，移动当前

窗口指针以显示当前活动的寄存器窗口。如果所有窗口都在使用中，将生成一个中断，并将最早的窗口（调用嵌套中最远的窗口）保存到内存中，保存的窗口指针标识最近保存在内存中的窗口。

全局变量

环形缓冲区用于处理**局部变量**。而对于**全局变量**，有两种处理方法：

1. 高级语言 (HLL) 中声明为全局的变量，由**编译器**为其分配内存位置，所有引用这些变量的机器指令都会使用**内存引用**操作数。
2. 使用一组固定数量的**全局寄存器**，可供所有过程使用。由**链接器**决定哪些全局变量分配到寄存器中。

寄存器与 Cache

下表展示了使用寄存器组和 Cache 的区别：

寄存器文件	Cache
保存**所有**局部标量变量	保存**最近使用的**局部标量变量
保存**单个变量**	保存**内存块**
由编译器决定全局变量	**最近使用的**全局变量
根据**过程嵌套深度**进行保存/恢复	根据 **Cache 替换算法**进行保存/恢复
寄存器寻址	**内存寻址**
一个周期内寻址和访问**多个**操作数	每个周期寻址和访问**一个**操作数

寄存器保留了所有局部标量变量，节省了时间，但浪费了空间，因为并非所有过程都需要分配完整窗口空间。相比之下 Cache 动态存储必要的数据，更有效利用了空间。

要访问基于窗口的寄存器组中的标量变量，需要给出窗口号和一个寄存器号，通过译码器得到对应寄存器；要访问 Cache 则需要一个完整的地址，对比标签位是否命中。

13.4 基于编译器的寄存器优化

HLL 不会显式引用寄存器，由**编译器**负责寄存器的优化。优化过程如下：

- 为每个候选变量分配符号寄存器或虚拟寄存器
- 映射符号寄存器到实际寄存器
- 不重叠的符号寄存器可以共享实体寄存器
- 如果没有实体寄存器，变量只能用内存

优化任务的本质是判断在任意时刻，哪些数据需要放到寄存器中。寄存器优化主要针对寄存器**较少**的情况。寄存器较多时，优化的效果不好。 RISC 编译器中最常用的技术是**图着色**。

图着色问题

给定一个包含节点和边的无向图，给每个节点分配一个颜色，相邻节点用不同颜色，求颜色数量最少的分配方式。将其对应到寄存器优化问题上就是：节点代表**符号寄存器**，边代表相连的

两个寄存器**在同一程序段中使用**，颜色数量即**实体寄存器数量**。如果颜色数量达到上限，不能被着色的节点需要放到内存中。

13.5 RISC 流水线

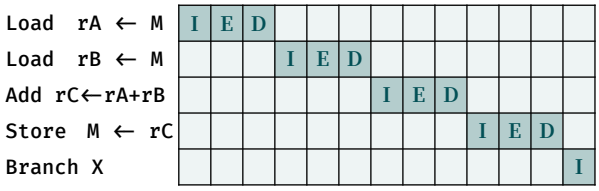
RISC 流水线中大部分指令是寄存器到寄存器的，一个指令周期分为两个阶段：

- I: Instruction Fetch 取指
- E: Execute 执行，由 ALU 操作寄存输入输出。

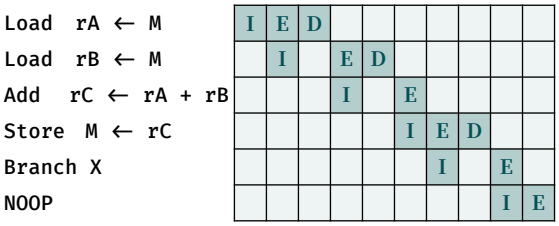
对于加载和保存操作，分为三个阶段：

- I: Instruction Fetch 取指
- E: Execute 执行，计算内存地址
- D: Memory 存储，**寄存器到内存**或**内存到寄存器**的操作

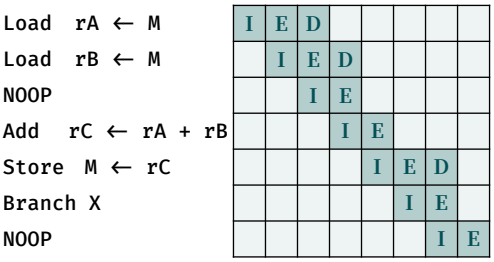
下图展示了几种时序：



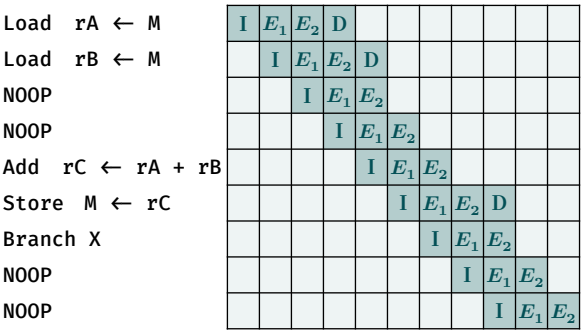
(a) 顺序执行



(b) 两阶段流水线



(c) 三阶段流水线



(d) 四阶段流水线

(a) 是没有使用流水线的**顺序执行**时序。(b) 是使用两阶段流水线的时序，由于同时只允许 1 个存储器访问，取指和存储会冲突，因此取指会延后一个周期。(c) 中支持 3 个存储器访问，使用三阶段流水线，考虑指令相关性：`Add rC ← rA + rB` 需要的操作数 `rB` 需要第二条指令结束，也就是第四个时钟周期才能得到，因此需要插入空指令 `NOOP` 。(d) 中把 E 阶段分为两个阶段，一个完成**寄存器的读**，另一个完成 **ALU 操作**和**寄存器的写**。

流水线优化

数据相关性和分支指令会影响流水线效率，解决方案是**延迟分支**和**循环展开**。

1. **延迟分支**

   分支直到其后一条指令执行后才生效。紧随分支的指令位置称为**延迟槽**。延迟槽中的指令与分支独立，不受分支影响，无论分支指令是否跳转，延迟槽中的那一条指令总是会执行。对于**无条件分支**，容易找到合适的指令填充延迟槽；对于**条件分支**，条件计算不能依赖于延迟槽的指令，不好填充延迟槽。

延迟分支对超标量**没有效果**，因为超标量中有多个延迟槽需要填充，同时要考虑指令相关性问题。一般使用分支预测。

2. **循环展开**

循环展开通过**多次复制循环体**提高性能。这种提升体现在如下方面：

- 降低循环开销
- 通过改进流水线性能增加**指令并行性**
- 增加寄存器、cache 或 TLB 的**局部性**

以下两段代码展示了循环展开的效果。这里把原书代码改写成了 C 语言描述。

```c
// 原始循环
for (int i = 2; i < n; i++) {
    a[i] = a[i] + a[i-1] * a[i+1];
}
```

```c
// 展开两次的循环
int i;
for (i = 2; i < n-1; i += 2) {
    a[i] = a[i] + a[i-1] * a[i+1];
    a[i+1] = a[i+1] + a[i] * a[i+2];
}
if (i == n-1) {
    a[n-1] = a[n-1] + a[n-2] * a[n];
}
```

第一个循环中，一个循环体访问了 3 次内存（`a[i]`，`a[i-1]`，`a[i+1]`）。第二个循环中，一个循环体执行了两个迭代操作。`a[i]` 和 `a[i+1]` 在循环体中使用了 2 次，也就是说，相当于每 2 次循环访问 4 次内存（`a[i]`，`a[i-1]`，`a[i+1]`，`a[i+2]`），减少了存储器访问的次数。

尽管在课件中没有涉及，原书第 10 版中还提到了**延迟加载**的方法。这种方法用于 `LOAD` 指令，类似于延迟分支，紧跟在 `LOAD` 指令之后的位置为**延迟槽**。处理器先执行 `LOAD`，然后继续执行**延迟槽**中的指令，后执行需要用到寄存器的指令。

13.6 RISC 和 CISC 的争议

评判 RISC 的方法分为两类：

- **定量 (Quantitative)**：比较程序大小和执行速度。
- **定性 (Qualitative)**：对高级语言的支持程度，以及对 VLSI 资源的优化使用。

然而，比较时会存在如下问题：

- 没有一对 RISC 和 CISC 有可比性

- 没有确定的**测试程序集**
- 难以区分**硬件效果**和**编译器效果**
- 对 RISC 的分析比较大多是在**模型机**而非商用机上进行
- 大多数商用设备是两者混合的

如今，两者已经相互融合。Intel 的 x86 系列融合了 RISC 的特点，增加了**寄存器数量**，同时强调**流水线技术**的设计。ARM 系列也越来越复杂。

14. 指令级并行与超标量处理器

14.1 超标量综述

回顾流水线的问题

设想一个理想流水线，指令分为 6 个阶段，每个时间单位有 1 个指令完成执行，指令数量足够多时执行效率为原来的 6 倍。然而实际流水线中，存在如下问题：

- 不同指令运行**步骤不同**，这导致部分阶段空闲。
- 不同阶段运行**时间不同**，这导致时间浪费。
- 指令独立性问题（数据、控制、资源相关性带来的冒险）。

  指令流水线**不是真正的并行**，尽管有多个指令同时处理，但它们不是同时进入流水线。

  进一步提高指令执行效率的方法：

- **超级流水线**
  - 原理：将原本的一个时钟周期细分，采用**双倍内部时钟速率**进行指令调度，在**一个外部时钟周期**内完成**两个阶段**。
  - 效果：可以获得两倍的指令吞吐量，类似于提升主频。
  - 局限性：仍然不是真正意义上的指令级并行，性能受限于时钟周期和指令阶段执行时间的短板。
- **超标量**

大多数指令是针对**标量**的操作（算术、加载、存储、条件分支）。超标量方法使这些指令可以**同时启动、独立执行**，是真正的指令并行处理技术。超标量的本质是能在不同的流水线中**独立并发**地执行指令，允许指令执行顺序与程序顺序不同。

传统标量的流水线组织：单流水线，通过让多条指令同时处于不同阶段实现并行。

超标量的流水线组织：多功能单元，每个单元都是流水线。利用多个功能单元并行执行指令流。硬件和编译器共同保证并行执行不违反程序意图。

下图展示了超标量流水线的时序：

	1	2	3	4	5	6	7	8	9
指令 1	FI	DI	CO	FO	EI	WO			
指令 2	FI	DI	CO	FO	EI	WO			
指令 3		FI	DI	CO	FO	EI	WO		
指令 4		FI	DI	CO	FO	EI	WO		
指令 5			FI	DI	CO	FO	EI	WO	
指令 6			FI	DI	CO	FO	EI	WO	
指令 7				FI	DI	CO	FO	EI	WO
指令 8				FI	DI	CO	FO	EI	WO

超标量的限制

超标量存在的设计问题：

- 流水线本身的相关性依然存在。
- 多流水线带来了更复杂的指令相关性问题。
- 需要编译器具备更复杂的优化技术。

指令级并行性 (Instruction-Level Parallelism, ILP) 是程序指令平均**并行执行的程度**。**指令间的相关性**是限制 ILP 的最重要原因，包括：

- **真实数据相关性** (True Data Dependency)

也叫写后读 (Read-after-Write, RAW)。必须等前一条指令执行完，后一条才能取操作数。这是真实的相关，必须遵守。例如：

```
ADD r1, r2  ; 写 r1
MOV r3, r1  ; 读 r1
```

这段代码中，第二条直到第一条执行完成之前都无法执行。

- **过程相关性** (Procedural Dependency)

即控制相关性。分支指令导致无法确定下一条指令的位置（特别是变长指令集）。 RISC 因为是定长指令，相对受影响较小，但仍存在分支预测问题。

- **资源冲突** (Resource Conflict)

多条指令竞争同一功能单元。 解决方法：**资源复制** (Duplication of Resources) 。

- **输出相关性** (Output Dependency)

也叫写后写 (Write-after-Write, WAW) 。两条指令写同一个寄存器。乱序执行时可能导致结果覆盖顺序错误。

- **反相关性** (Anti-dependency)

也叫读后写 (Write-after-Read, WAR) 。后一条指令写前一条指令要读的寄存器。若乱序执行，后一条先写，前一条就读到了错误的新值。

14.2 超标量设计问题

指令并行性和机器并行性

限制并行的因素：

- 指令级并行性 (ILP)：程序指令平均并行执行的程度。
- 编译器能力：编译器挖掘并行的能力。
- 硬件技术：硬件支持并行运行的能力。

以下两段指令展示了指令级并行性：

```
// 独立，可以并行执行
r1 ← r2
r3 ← r3 + 1
r4 ← r4 + r2
```

```
// 存在依赖，不能并行执行
r3 ← r3 + 1
r4 ← r3 + 2
[r4] ← r0
```

机器并行性 (Machine Parallelism) 是指**利用指令级并行性的能力程度**，取决于同时能取指和执行的指令数（并行流水线数量）和处理器发现独立指令所用机制的速度和复杂性。指令级并行性和机器并行性都是提高性能的重要因素。

指令发射策略

指令发射 (Instruction Issue) 是指启动指令去处理器功能单元执行的过程。为了提高并行性，处理器会改变指令的发射和完成顺序。指令发射涉及到三个顺序：取指顺序、执行顺序、修改寄存器和内存的顺序。超标量指令发射策略分为三种：

1. **按序发射，按序完成**：按照指令发生的顺序发射指令，以同样的顺序写回结果。这种策略效率低下，一旦功能单元冲突，指令发射必须等待。
2. **按序发射，乱序完成**：按顺序发射，允许执行快的指令先完成。由于执行顺序和原来不一样，可能导致**输出相关性**问题。比如：

   ```
   r1 ← r1 + r2
   r1 ← r2 + 1   // 执行速度更快，先执行完会影响第一条的写
   ```

   解决方法是**动态调度**策略，将**有依赖关系的指令**和**独立指令**分离，监控前者的关联值，如果所有关联值可用，就将其发射。
3. **乱序发射，乱序完成**：将译码和执行**解耦**。
   - 译码：译码阶段可以持续取指和译码，译码后的指令放到缓冲区中。这个缓冲区是一个寄存器窗口，也被称为**指令窗口**。指令窗口需要足够大（通常超过 8 个）。
   - 执行：当功能单元可用时发射指令，不必按程序顺序。

   乱序发射可能导致**反相关性**问题。比如：

   ```
   r1 ← r2 + 1
   r2 ← r3 + 1   // 先执行完会影响第一条的读
   ```

寄存器重命名

反相关和**输出相关**本质上是寄存器数量有限导致的名字冲突，而非真正的数据依赖。解决方案是**寄存器重命名**，但无法解决真相关（写后读）问题。看一个例子：

```
// 原始指令
I1: R1 ← R1 + R2
I2: R3 ← R1 + 1
I3: R1 ← R2 + 1
I4: R4 ← R1 + R3
```

```
// 寄存器重命名
I1: R1b ← R1a + R2a
I2: R3b ← R1b + 1
I3: R1c ← R2a + 1
I4: R4  ← R1c + R3b
```

原始指令中的相关性如下：

- 真相关： I1 和 I2 、I3 和 I4
- 反相关： I2 和 I3
- 输出相关： I1 和 I3

经过寄存器重命名后，反相关和输出相关被消除，I3 可以立即发射，但是真相关**无法消除**。

提高性能的技术与控制相关性

提高超标量性能的三种技术分别是**资源复制**、**乱序发射**和**寄存器重命名**，三者相互影响。另外，第 13 章中提到 RISC 常用**延迟分支**方法解决控制相关性问题，但这种方法对超标量没有效果，因为需要填充的延迟槽太多，同样会引起相关性问题。**分支预测**在超标量中最常用。

超标量的执行过程

1. 静态程序通过**取指**和**分支预测**，形成动态的指令流。
2. 指令流经过处理器的**相关性检查**，会去掉不必要的相关性，比如反相关和输出相关。然后将指令放到执行窗口中，等待执行。
3. 在执行窗口中的指令，根据**真相关性**来排序。处理器根据真相关性和资源可用性，发射指令到执行单元。
4. 最后的执行结果需要**提交** (commiting) ，因为指令不是按照原有的顺序来执行的，同时分支预测和推测执行使得有些执行的结果需要丢弃。

超标量的实现

超标量的实现有以下关键要素：

- 同时取多条指令，需要多个**取指**和**译码**流水阶段，以及**分支预测**。
- 确定涉及寄存器值**真相关**的逻辑，以及真相关指令的位置。
- 处理不必要的相关性：**反相关**和**输出相关**。
- 并行启动多条指令的机制：**指令窗口**和**乱序发射**。
- 用于并行执行多条指令的资源，包括**多个流水功能单元**，以及可供多个同时访问的**存储器层次结构**。
- 按正确顺序提交处理状态的机制。

附录 A: 24 级期中试题回忆

说明

最后两道大题是完整原题，其他大题只有大概题意。笔者期中也就 60 多分。前面选择和判断题太多了就没记。

简答题

1. Amdahl 定律及其定义
2. 画出包含中断的指令周期状态图
3. 描述 Cache 的几种替换算法，并根据不同映射方式进行分类讨论
4. 描述计算机处理中断的全过程，以及中断结束后计算机如何处理
5. Hamming 码的数据位数 m 和 纠错位数 k 的关系，计算 16 个数据位需要多少纠错位
6. 写出 I/O 的核心功能，解释为什么说 I/O 是主机和外设的桥梁

大题

1. 给了一个内存单元的表格和 PC 的值，并给了两个操作码和一段程序，写出执行过程中各寄存器 (PC, IR, AC) 值的变化
2. 给出内存和 Cache 的数据，写出组相联映射方式下的地址格式，计算十六进制数 2025A 对应的行号
3. 将 256K * 4 位的存储芯片组织成一个 2 MB 的内存，计算需要的地址线数量并画出设计图
4. 一个硬盘有 4 个面，每个面上有 512 条磁道，每个磁道分为 64 个扇区，每个扇区的大小为 512B ，该磁盘平均寻道时间为 3ms ，相邻磁道的移动时间为 0.1ms ，磁盘转速为 18000 转/分钟，文件采用柱面的形式存储
    1. 磁盘的总容量为多大?
    2. 平均存取时间是多少?
    3. 一个大小为 2KB 的文件，随机存储在不同磁道和扇区，计算该文件的读取时间。
    4. 若不考虑总线速度限制，只考虑磁盘读取速度，传送一个 1MB 的文件需要多长时间?
5. 某计算机采用页式存储管理方式，相关参数如下：
    - 逻辑地址空间大小： 32 位
    - 物理地址空间大小： 28 位
    - 页面大小： 4KB
    - 页表项结构：仅包含物理块号和有效位（用于标识该页是否已调入物理内存，占 1 位）

    回答以下问题：
    1. 计算该系统中逻辑地址的页号位数和页内偏移位数，并画出逻辑地址的结构，标明各部分位数
    2. 计算单个页表项的位数和整个页表的总大小（单位 KB ，保留 1 位小数）
    3. 某进程的页表部分内容如下表（自行想象），有效位 1 表示该页已在内存， 0 表示未在内存，请将逻辑地址 0x00012345H 转换为对应的物理地址

附录 B: 24 级期末试题回忆

说明

记得去听最后一节课，老师会划范围和重点。多看课后习题。选择和判断各 15 道，选择一题 2 分，判断一题 1 分。简答一道 5 分。大题一共 30 分，其中最后一道 10 分。

简答题（均为教材课后习题）

1. 内存映射 I/O 和独立编址 I/O 的区别
2. 机器指令中的操作数类型有哪些
3. 条件码的功能
4. RISC 指令集的特征
5. 如何解读水平微指令

大题

1. 给出内存和 Cache 的数据，写出直接映射和组相联映射的地址格式，计算 1234567H 在两种映射方式下分别对应的行号（和组号）
2. 给一段 Hamming 码，对其纠错
3. 写出 11001011 在无符号、原码、反码、补码、移码表示法下对应的值
4. 给出基址寄存器、变址寄存器、程序计数器和比例因子，给一张内存内容的表，分别写出以下通过寻址方式取到的值:
    1. 直接寻址
    2. 间接寻址
    3. 基址寻址
    4. 相对寻址
    5. 前变址寻址
5. （习题 16.6 ）给出一个有两阶段流水线的超标量处理器，具体结构可参照教材。如下代码对应的指令都有取指、译码、执行、存储（除了 I1 和 I5 ）阶段，其中加法操作需要 2 个周期，乘法需要 3 个周期，其他阶段需要 1 个周期。
    1. 写出指令中存在的依赖关系
    2. 现使用寄存器重命名消除了反相关和输出相关，填写按序发射按序完成的时序图
    3. 填写按序发射乱序完成的时序图
    4. 填写乱序发射乱序完成的时序图，其中有一个能缓存 6 条指令的窗口
    5. 计算串行执行这段程序需要的周期数，并计算乱序发射乱序完成的加速比

```
I1: MOV R1, A   ; R1 ← Memory (A)
I2: ADD R2, R1  ; R2 ← (R2) + (R1)
I3: ADD R3, R4  ; R3 ← (R3) + (R4)
I4: MUL R4, R5  ; R4 ← (R4) * (R5)
I5: CMP R6      ; R6 ← (R6)
I6: MUL R6, R7  ; R6 ← (R6) * (R7)
```